

March 11, 2026

1 SARSA en CliffWalking-v1

Estudio del algoritmo de Diferencia Temporal (TD) On-Policy

Este notebook implementa y analiza el algoritmo **SARSA** (State-Action-Reward-State-Action) sobre el entorno **CliffWalking-v1** de Gymnasium. A diferencia de los métodos de Monte Carlo, SARSA utiliza **bootstrapping**, permitiendo al agente aprender de sus estimaciones actuales sin esperar al final del episodio.

1.0.1 SARSA

- **Aprendizaje On-Policy:** El agente aprende sobre la política que está ejecutando actualmente (incluyendo la exploración ϵ -greedy).
- **Diferencia Temporal (TD):** Actualiza los valores Q basándose en la recompensa inmediata y la estimación del siguiente estado-acción:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- **Comportamiento en CliffWalking-v1:** Al ser on-policy, SARSA aprende bajo la política ϵ -greedy que realmente ejecuta. Esto significa que el agente tiene en cuenta el riesgo de caer al acantilado durante la exploración, lo que le lleva a preferir una **ruta segura** alejada del borde (17 pasos) frente a la ruta óptima más corta (13 pasos) que bordea el acantilado.

1.1 1. Preparación del Entorno

La preparación consta de las siguientes partes: - **Instalación de dependencias:** librería gymnasium. - **Importación de librerías:** numpy, matplotlib, tqdm. - **Creación del entorno CliffWalking-v1:** espacio de observación `Discrete(48)`, espacio de acciones `Discrete(4)` (N, E, S, W), `render_mode='ansi'` para visualización en texto. El grid es de 4×12 casillas; la fila inferior contiene el acantilado (estados 37–46) con penalización -100. - **Importación de clases del repositorio:** `SARSAAgent`, funciones de visualización.

1.2 ##### Código de la Instalación e Importación

```
[1]: %%capture
    #@title Instalamos gymnasium
    !pip install gymnasium
```

```
[19]: #@title Importamos librerias
import sys
```

```

sys.path.insert(0, '../src')

import os
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm
import gymnasium as gym

from agents import SARSAAgent
from plotting import plot, plot_lengths, show_greedy_episode, print_q_summary,
    plot_comparison, plot_lengths_comparison

```

```

[20]: # @title Semilla para reproducibilidad (sección 5.4 del PDF)
# Patrón recomendado: fijar la misma semilla en NumPy, Python y Gymnasium.
# Sin PyTorch porque este notebook es tabular (no usa redes neuronales).
SEED = 2024

# Fijar la semilla en NumPy
np.random.seed(SEED)
np.random.default_rng(SEED)

# Fijar la semilla en Python (evita variabilidad en hashing)
os.environ['PYTHONHASHSEED'] = str(SEED)

print(f"Semilla fija: SEED = {SEED}")

```

Semilla fija: SEED = 2024

```

[21]: #@title Importamos el entorno CliffWalking-v1
env = gym.make('CliffWalking-v1', render_mode='ansi')
env.reset(seed=SEED) # Fija la semilla del entorno (patrón sección 5.4 del
    PDF)

nS = env.observation_space.n # 48
nA = env.action_space.n # 4
print(f"Estados: {nS}, Acciones: {nA}")
print(f"Recompensas: -1 por paso | -100 caída al acantilado | 0 al llegar")
print()
# # Estado inicial de ejemplo
# obs, info = env.reset(seed=SEED)
# print(f"Estado inicial (seed={SEED}): {obs}")
# taxi_row, taxi_col, pass_loc, dest_idx = env.unwrapped.decode(obs)
# locs = ['R', 'G', 'Y', 'B', 'en taxi']
# print(f" Taxi: ({taxi_row},{taxi_col}), Pasajero: {locs[pass_loc]}, Destino:
    {locs[dest_idx]}")
# print(env.render())

```

Estados: 48, Acciones: 4

Recompensas: -1 por paso | -100 caída al acantilado | 0 al llegar

1.3 2. Diseño del Agente

El diseño del agente sigue el esquema de Gymnasium (sección 5.2): `__init__`, `get_action`, `update`, `stats`.

1.3.1 Política Epsilon-Greedy

- El agente utiliza una política ϵ -soft para garantizar la exploración continua del espacio de estados.
- Con probabilidad $1 - \epsilon$, selecciona la acción con el mayor valor Q (explotación).
- Con probabilidad ϵ , selecciona una acción al azar (exploración).

1.3.2 Algoritmo SARSA

- Implementado en la clase `SARSAAgent`, este algoritmo actualiza la tabla Q en cada paso del entorno.
- A diferencia de Q-Learning, SARSA es más conservador ya que considera la acción real que el agente tomará en el siguiente paso (la cual podría ser una acción exploratoria arriesgada).

```
[5]: UP, RIGHT, DOWN, LEFT = 0,1,2,3
ACTION_NAMES = {0: 'N↑', 1: 'E→', 2: 'S↓', 3: 'W←', 4: 'PU', 5: 'DO'}
# La política greedy se obtiene con agent.pi_star(seed=SEED,
↪action_names=ACTION_NAMES)
```

1.4 3. Experimentación

1.4.1 3.1 Representaciones Gráficas

Gráfica 1 — Recompensa media acumulada: $f(t) = \frac{\sum_{i=1}^t R_i}{t}$. En CliffWalking-v1, los primeros episodios con ϵ alto acumulan muchas caídas al acantilado (-100 cada una), lo que hunde el promedio durante las fases iniciales. Al ser acumulada, puede estabilizarse en valores muy negativos; en este run la media final ronda **-8025** aunque el episodio greedy sea de -17.

Gráfica 2 — Longitud de los episodios: Mide cuántos pasos tarda el agente en llegar al objetivo o caer al acantilado. 1. **Inicio:** Los episodios son muy cortos porque el agente aleatorio cae al acantilado rápidamente (-100 y reset). 2. **Aprendizaje:** La longitud se estabiliza conforme el agente evita el acantilado y aprende a rodear por la fila superior. 3. **Convergencia:** La política greedy es de 17 pasos (ruta segura por la fila superior), pero la longitud media final del entrenamiento queda alrededor de **19 pasos** en este run.

Función `show_greedy_episode`: Permite observar el comportamiento final del agente sin exploración, verificando si ha aprendido a navegar desde el estado inicial (36, esquina inferior izquierda) hasta el objetivo (47, esquina inferior derecha) evitando el acantilado.

1.4.2 3.2 SARSA en CliffWalking-v1

Se entrenan **10 000 episodios** con ε con decaimiento: $\varepsilon = \min(1.0, 1000/(t + 1))$, igual que en Q-Learning y MC para poder comparar después resultados.

El decaimiento es necesario porque: - Al inicio, ε alto favorece una exploración amplia (el agente aprende a no caer al acantilado a base de penalizaciones de -100). - Al final, $\varepsilon \rightarrow 0$ hace que la política de comportamiento se acerque a la greedy, produciendo episodios más cortos y estables.

La semilla (SEED=2024) garantiza reproducibilidad. Se fija tanto en NumPy como en el entorno.

```
[22]: # @title Aprendizaje on-policy
agent_S = SARSAAgent(env, epsilon=1, alpha=0.1, discount_factor=0.99,
    ↪decay=True)
agent_S.train(num_episodes=10000)
list_stats_S, list_lengths_S = agent_S.stats()
Q_S = agent_S.q_values
```

```
10%|          | 1002/10000 [01:23<12:16, 12.21it/s]
success: -64704.6510, epsilon: 1.0000

24%|          | 2409/10000 [02:38<00:06, 1244.12it/s]
success: -40001.0910, epsilon: 0.5000

34%|          | 3381/10000 [02:38<00:02, 2210.53it/s]
success: -26687.4550, epsilon: 0.3333

45%|          | 4534/10000 [02:38<00:01, 2771.26it/s]
success: -20025.3777, epsilon: 0.2500

55%|          | 5481/10000 [02:39<00:01, 3042.33it/s]
success: -16026.9394, epsilon: 0.2000

65%|          | 6519/10000 [02:39<00:01, 3327.05it/s]
success: -13360.3207, epsilon: 0.1667

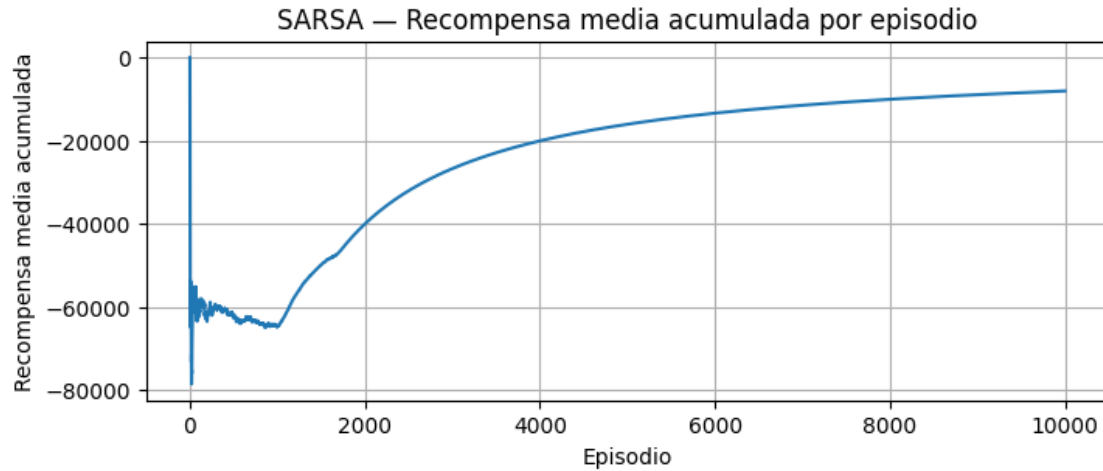
76%|          | 7553/10000 [02:39<00:00, 3398.30it/s]
success: -11455.1813, epsilon: 0.1429

86%|          | 8612/10000 [02:40<00:00, 3271.16it/s]
success: -10026.4093, epsilon: 0.1250

97%|          | 9718/10000 [02:40<00:00, 3578.07it/s]
success: -8914.9161, epsilon: 0.1111

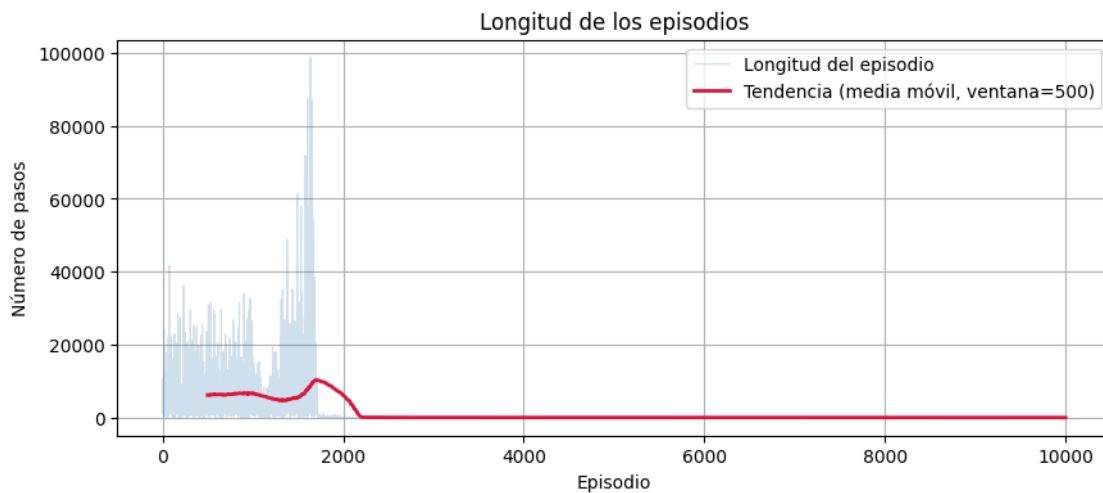
100%|         | 10000/10000 [02:40<00:00, 62.30it/s]
success: -8025.6447, epsilon: 0.1000
```

```
[7]: #@title Recompensa media acumulada por episodio (on-policy)
plot(list_stats_S, title='SARSA - Recompensa media acumulada por episodio',
     ylabel='Recompensa media acumulada')
print(f'Recompensa media final: {list_stats_S[-1]:.2f}')
```



Recompensa media final: -8025.64

```
[8]: #@title Longitud de episodios (on-policy)
plot_lengths(list_lengths_S)
print(f'Longitud media final (últimos 1000 episodios): {np.
     mean(list_lengths_S[-1000:]):.2f} pasos')
```



Longitud media final (últimos 1000 episodios): 19.33 pasos

Con **148 de 192 entradas visitadas** (77.1%), Max=0, Min=-289.398 y Mean=-49.794. El valor máximo de 0 corresponde al estado objetivo. Los valores muy negativos de Min y Mean reflejan el impacto de las caídas al acantilado (-100) durante el entrenamiento on-policy: SARSA actualiza Q con las acciones que realmente ejecuta (incluyendo exploración), por lo que los estados próximos al acantilado acumulan valores mucho más negativos que en Q-Learning (-112 de mínimo).

```

--- On-Policy - Tabla Q ---
Entradas no nulas : 148/192 (77.1%)
Max Q               : 0.000
Min Q (no nulo)    : -289.398
Q media (no nulo)   : -49.794

```

SARSA, al ser on-policy, aprende los valores Q bajo la política exploratoria ϵ -greedy. Esto hace que los estados cercanos al acantilado tengan valores penalizados por las caídas durante el entrenamiento, empujando la política greedy resultante a preferir la **ruta segura**: $N \uparrow \times 3 \rightarrow E \rightarrow 11 \rightarrow S \downarrow \times 3$ (17 pasos, recompensa -17), alejada del borde.

```
[10]: # @title Política final (on-policy)
pi, actions = agent_S.pi_star(seed=SEED, action_names=ACTION_NAMES)
print("Política óptima obtenida (on-policy)")
print(f"Acciones: {actions}")
print()
show_greedy_episode(env, Q_S, seed=SEED, title="SARSA - Episodio greedy")
```

Política óptima obtenida (on-policy)
Acciones: N↑ → N↑ → N↑ → E→ → E→ → E→ → E→ → E→ → E→ → E→ → E→ → E→ → E→ → E→ → E→ →
S↓ → S↓ → S↓

SARSA - Episodio greedy | Recompensa: -17 | Pasos: 17 | Terminado

Estado inicial:

[illegible]

6

```

o o o o o o o o o o o o
o o o o o o o o o o o o
o o o o o o o o o o o o
o C C C C C C C C C C x

```

```

[11]: # state, info = env.reset(seed=SEED)
      # # Extraer: taxi_row, taxi_col, pass_loc, dest_idx
      # _, _, pass_loc, dest_idx = env.unwrapped.decode(state)

      # plot_policy_taxi(env, agent_S.q_values, passenger_loc=pass_loc,
      ↪ destination_idx=dest_idx)

```

1.4.3 3.3 Modificación de parámetros de SARSA

Los hiperparámetros en el algoritmo SARSA son fundamentales para equilibrar la explotación versus la exploración y cómo se gestionan las nuevas experiencias.

- La tasa de aprendizaje (α) controla qué tan rápido el agente actualiza sus valores Q ante nueva información; un valor elevado puede acelerar la convergencia inicial pero generar inestabilidad, mientras que un valor bajo asegura una convergencia más suave y robusta.
- El parámetro épsilon (ϵ) determina el grado de exploración. En CliffWalking, la exploración tiene un coste especialmente alto (caídas al acantilado de -100), por lo que reducir ϵ prematuramente puede hacer que el agente aprenda una política subóptima, mientras que mantenerlo alto retrasa la convergencia y acumula más penalizaciones.
- El factor de descuento (γ) dicta la importancia de las recompensas futuras frente a las inmediatas. En CliffWalking, un γ cercano a 1.0 es esencial para que el agente valore alcanzar el objetivo (recompensa 0 en el estado terminal) pese al coste de -1 por paso. Reducir γ hace que el agente pierda perspectiva del largo plazo y no pueda planificar la ruta correctamente.

```

[12]: # @title Aprendizaje on-policy
agent_S = SARSAAgent(env, epsilon=0.01, alpha=0.1, discount_factor=0.99,
↪ decay=False)
agent_S.train(num_episodes=10000)
list_stats_S_epsilon, list_lengths_S_epsilon = agent_S.stats()
Q_S = agent_S.q_values

```

```

19%|          | 1902/10000 [00:00<00:01, 4383.35it/s]
success: -26.7700, epsilon: 0.0100
success: -21.1900, epsilon: 0.0100

39%|          | 3884/10000 [00:00<00:01, 4829.24it/s]
success: -19.2863, epsilon: 0.0100

48%|          | 4846/10000 [00:01<00:01, 4745.95it/s]
success: -18.2695, epsilon: 0.0100

```

```

58%|          | 5818/10000 [00:01<00:00, 4746.88it/s]
success: -17.7008, epsilon: 0.0100
success: -17.3360, epsilon: 0.0100

77%|          | 7709/10000 [00:01<00:00, 4696.81it/s]
success: -17.0886, epsilon: 0.0100

87%|          | 8662/10000 [00:01<00:00, 4730.23it/s]
success: -16.8810, epsilon: 0.0100
success: -16.6926, epsilon: 0.0100

100%|         | 10000/10000 [00:02<00:00, 4558.62it/s]
success: -16.5619, epsilon: 0.0100

```

```

[13]: # @title Aprendizaje on-policy
agent_S = SARSAAgent(env, epsilon=0.2, alpha=0.5, discount_factor=0.99,
    ↪decay=True)
agent_S.train(num_episodes=10000)
list_stats_S_alpha, list_lengths_S_alpha = agent_S.stats()
Q_S = agent_S.q_values

```

```

10%|          | 1001/10000 [01:19<13:23, 11.19it/s]
success: -63814.6530, epsilon: 1.0000

21%|          | 2133/10000 [01:27<00:07, 1002.89it/s]
success: -34029.8805, epsilon: 0.5000

33%|          | 3276/10000 [01:28<00:04, 1678.50it/s]
success: -22717.7577, epsilon: 0.3333

43%|          | 4335/10000 [01:28<00:02, 2078.78it/s]
success: -17053.2940, epsilon: 0.2500

53%|          | 5331/10000 [01:28<00:01, 2368.25it/s]
success: -13651.4592, epsilon: 0.2000

63%|          | 6289/10000 [01:29<00:01, 2216.96it/s]
success: -11383.6522, epsilon: 0.1667

74%|          | 7419/10000 [01:29<00:00, 2714.58it/s]
success: -9762.5647, epsilon: 0.1429

84%|          | 8379/10000 [01:30<00:00, 2716.27it/s]
success: -8545.8446, epsilon: 0.1250

93%|          | 9302/10000 [01:30<00:00, 2939.17it/s]

```



```

success: -7599.9862, epsilon: 0.1111
100%|      | 10000/10000 [01:30<00:00, 110.22it/s]
success: -6843.2892, epsilon: 0.1000

```

```

[16]: # @title Aprendizaje on-policy
agent_S = SARSAgent(env, epsilon=0.2, alpha=0.1, discount_factor=0.25,
    ↪decay=True)
agent_S.train(num_episodes=11000)
list_stats_S_discount, list_lengths_S_discount = agent_S.stats()
Q_S = agent_S.q_values

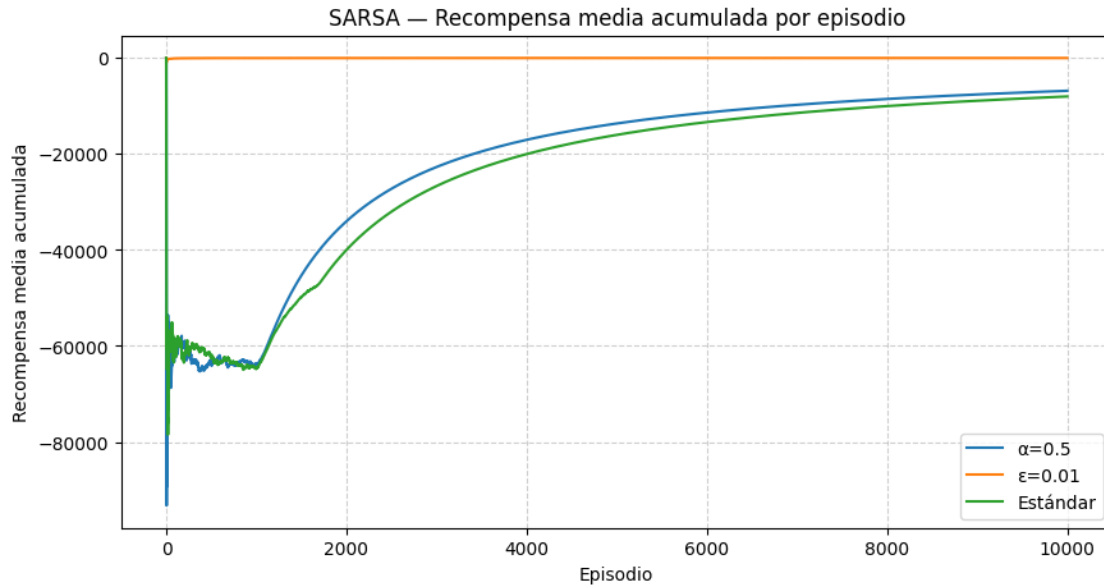
```

```

0%|      | 0/11000 [00:00<?, ?it/s] 10%|      | 1104/11000
[01:33<07:36, 21.67it/s]
success: -65078.9236, epsilon: 0.9091
20%|      | 2203/11000 [01:51<03:12, 45.78it/s]
success: -34261.4523, epsilon: 0.4545
30%|      | 3301/11000 [02:19<05:15, 24.42it/s]
success: -23542.7670, epsilon: 0.3030
40%|      | 4407/11000 [03:06<03:51, 28.49it/s]
success: -18530.3059, epsilon: 0.2273
50%|      | 5525/11000 [04:58<06:38, 13.74it/s]
success: -16503.9244, epsilon: 0.1818
60%|      | 6593/11000 [05:53<07:36, 9.66it/s]
success: -14437.0356, epsilon: 0.1515
70%|      | 7717/11000 [07:23<00:34, 93.98it/s]
success: -13330.0258, epsilon: 0.1299
80%|      | 8808/11000 [09:53<02:02, 17.95it/s]
success: -13082.7717, epsilon: 0.1136
90%|      | 9908/11000 [11:44<00:47, 23.15it/s]
success: -12555.1994, epsilon: 0.1010
100%|     | 11000/11000 [13:29<00:00, 13.59it/s]
success: -12082.2914, epsilon: 0.0909

```

```
[17]: #@title Recompensa media acumulada por episodio (on-policy)
lista_plots_recompensa = [list_stats_S_alpha, list_stats_S_epsilon,
    ↪ list_stats_S]
labels = ['=0.5', '=0.01', 'Estándar']
plot_comparison(lista_plots_recompensa, title='SARSA - Recompensa media
    ↪ acumulada por episodio', ylabel='Recompensa media acumulada', labels=labels)
for lista, label in zip(lista_plots_recompensa, labels):
    print(f'Recompensa media final de {label}: {lista[-1]:.2f}')
```



Recompensa media final de $\alpha=0.5$: -6843.29
 Recompensa media final de $\epsilon=0.01$: -16.56
 Recompensa media final de Estándar: -8025.64

```
[18]: #@title Longitud de episodios (on-policy)
lista_plots_longitud = [list_lengths_S_alpha, list_lengths_S_epsilon,
    ↪ list_lengths_S]
plot_lengths_comparison(lista_plots_longitud, labels=labels)
for lista, label in zip(lista_plots_longitud, labels):
    print(f'Longitud media final (últimos 1000 episodios) de {label}: {np.
    ↪ mean(lista[-1000:]):.2f} pasos')
```



Longitud media final (últimos 1000 episodios) de $\alpha=0.5$: 26.88 pasos
 Longitud media final (últimos 1000 episodios) de $\epsilon=0.01$: 15.19 pasos
 Longitud media final (últimos 1000 episodios) de Estándar: 19.33 pasos

1.4.4 3.3.1 Análisis de los hiperparámetros

1.4.5 3.3.1 Análisis de Resultados y Comparación de Configuraciones

En esta sección se evalúa el impacto de los hiperparámetros en el aprendizaje de SARSA sobre CliffWalking-v1. Se han definido dos configuraciones específicas (más la original) para observar cómo cada parámetro afecta al entrenamiento:

1. **Configuración 1** ($\epsilon = 0.01$) sin decaimiento: Minimiza la exploración desde el inicio. En CliffWalking, esto sugiere que el agente apenas cae al acantilado, lo que le permite acumular recompensas menos negativas rápidamente. La recompensa media final es de **-16.56**, muy próxima a la ruta greedy de -17, confirmando que aprende la política segura con pocas perturbaciones.
2. **Configuración 2** ($\alpha = 0.5$): Aumenta la tasa de aprendizaje, lo que puede acelerar la absorción de experiencias de penalización severa (-100), pero también introduce más ruido en las actualizaciones. La recompensa media final es de **-6843.29**, mejor que la estándar pero muy lejos de $\epsilon = 0.01$; además, muestra episodios más largos (~27 pasos), lo que indica una política menos eficiente. #### **Interpretación de las Gráficas**

Recompensa Media Acumulada: - La configuración con $\epsilon = 0.01$ fijo obtiene la mejor recompensa media final (**-16.56**). Al explorar muy poco, el agente evita prácticamente todas las caídas al acantilado desde el inicio, aprendiendo rápidamente a seguir la ruta segura. En CliffWalking, a diferencia de entornos con estructuras de recompensa complejas, la exploración mínima es suficiente porque el objetivo es evitar una penalización obvia (-100). - La configuración estándar obtiene una recompensa media de **-8025.64**, muy negativa debido a las caídas acumuladas en los primeros episodios con $\epsilon \approx 1$. La media acumulada se ve dominada por las fases iniciales de exploración

ciega, donde cada episodio puede acumular decenas de caídas al acantilado. - La configuración con $\alpha = 0.5$ obtiene **-6843.29**, mejor que la estándar, pero muy lejos de $\varepsilon = 0.01$. Un α más alto amplifica el impacto de las penalizaciones de -100, causando oscilaciones más pronunciadas y dificultando la estabilización de los valores Q .

Longitud de los Episodios: - Con $\varepsilon = 0.01$, la longitud final es ~15 pasos, muy cercana a la ruta segura de 17. - La configuración estándar se estabiliza alrededor de ~19 pasos. - Con $\alpha = 0.5$ la longitud final sube a ~27 pasos, señal de una política menos eficiente.

1.4.6 Guardar métricas (SARSA)

Esta celda guarda las series para poder comparar con otros notebooks.

```
[24]: import os, numpy as np
os.makedirs('metrics', exist_ok=True)
np.savez('metrics/sarsa_cliff.npz', list_stats=list_stats_S,
        list_lengths=list_lengths_S)
print('Guardado: metrics/sarsa_cliff.npz')
```

Guardado: metrics/sarsa_cliff.npz

1.5 4. Análisis y Conclusiones

1.5.1 4.1 Comparativa de Desempeño frente a Monte Carlo

SARSA converge a una política compatible con la óptima en CliffWalking-v1 de forma notablemente más rápida que los métodos de Monte Carlo. Al actualizar Q en cada paso, el agente incorpora inmediatamente las penalizaciones de -100 por caer al acantilado, sin necesidad de esperar al final del episodio como requiere MC.

Sin embargo, la política aprendida por SARSA no coincide con la óptima matemática (13 pasos de Q-Learning). Al ser on-policy, SARSA evalúa los estados bajo la política ε -greedy que realmente ejecuta: los estados próximos al acantilado reciben penalizaciones frecuentes durante el entrenamiento, lo que eleva sus valores negativos y lleva a la política greedy resultante a preferir la **ruta segura** de 17 pasos ($N \rightarrow E \rightarrow S \rightarrow S$), alejada del borde.

La tabla Q tiene **148 de 192 entradas visitadas** (77.1%), con Max=0, Min=-289.398 y Mean=-49.794. El mínimo mucho más negativo que en Q-Learning (-112) refleja que SARSA entrena explícitamente con caídas al acantilado, registrando los costes acumulados de múltiples penalizaciones de -100 en los estados adyacentes.

Al observar un episodio greedy, el agente rodea el acantilado por la fila superior, confirmando que ha aprendido a priorizar seguridad frente a brevedad como consecuencia de su naturaleza on-policy.

1.5.2 4.2 ¿Por qué SARSA aprende la ruta segura y no la óptima?

La diferencia fundamental respecto a Q-Learning reside en qué política se usa para la actualización de Q :

- **Q-Learning (off-policy):** actualiza con $\max_a Q(s', a)$ — siempre usa la acción greedy, independientemente de si durante el entrenamiento el agente cayó al acantilado por exploración.

- **SARSA (on-policy)**: actualiza con $Q(s', a')$ donde a' es la acción realmente ejecutada. Si esa acción fue exploratoria y resultó en una caída (-100), ese coste entra directamente en la actualización.

Como resultado, los valores Q de los estados junto al acantilado quedan mucho más penalizados en SARSA, y la política greedy final evita esa zona aunque sea el camino más corto. Este fenómeno es el **Ejemplo 6.6 de Sutton & Barto (2018)** y representa una de las diferencias más ilustrativas entre algoritmos on-policy y off-policy en la literatura de RL.

1.5.3 4.3 Propuestas Futuras

- Comparar directamente con **Q-Learning** sobre CliffWalking para visualizar la brecha de rendimiento greedy (13 vs 17 pasos) y el trade-off entre política óptima y política segura.
- Evaluar el impacto de distintos valores de ε fijo en la distancia de la ruta aprendida al acantilado: a mayor exploración, más alejada la ruta; a menor exploración, más cercana al borde.